



Programación de Servicios y Procesos

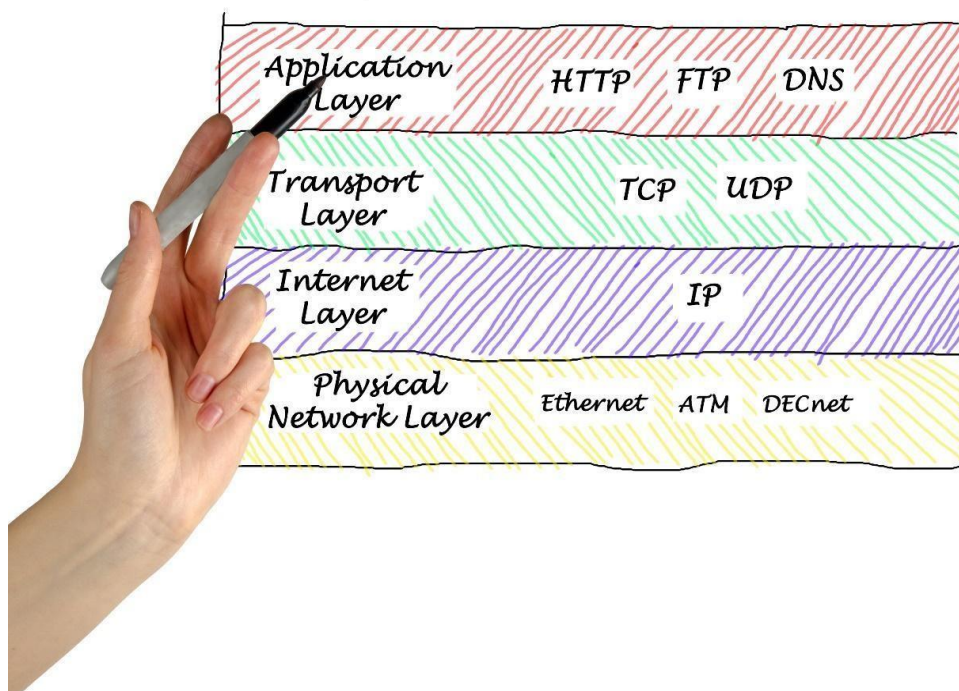
Resumen. Programación de comunicaciones en red

1.1. Comunicación entre aplicaciones

Las redes de ordenadores están formadas por un conjunto de dispositivos que se encuentran conectados entre sí para poder intercambiar información entre ellos.

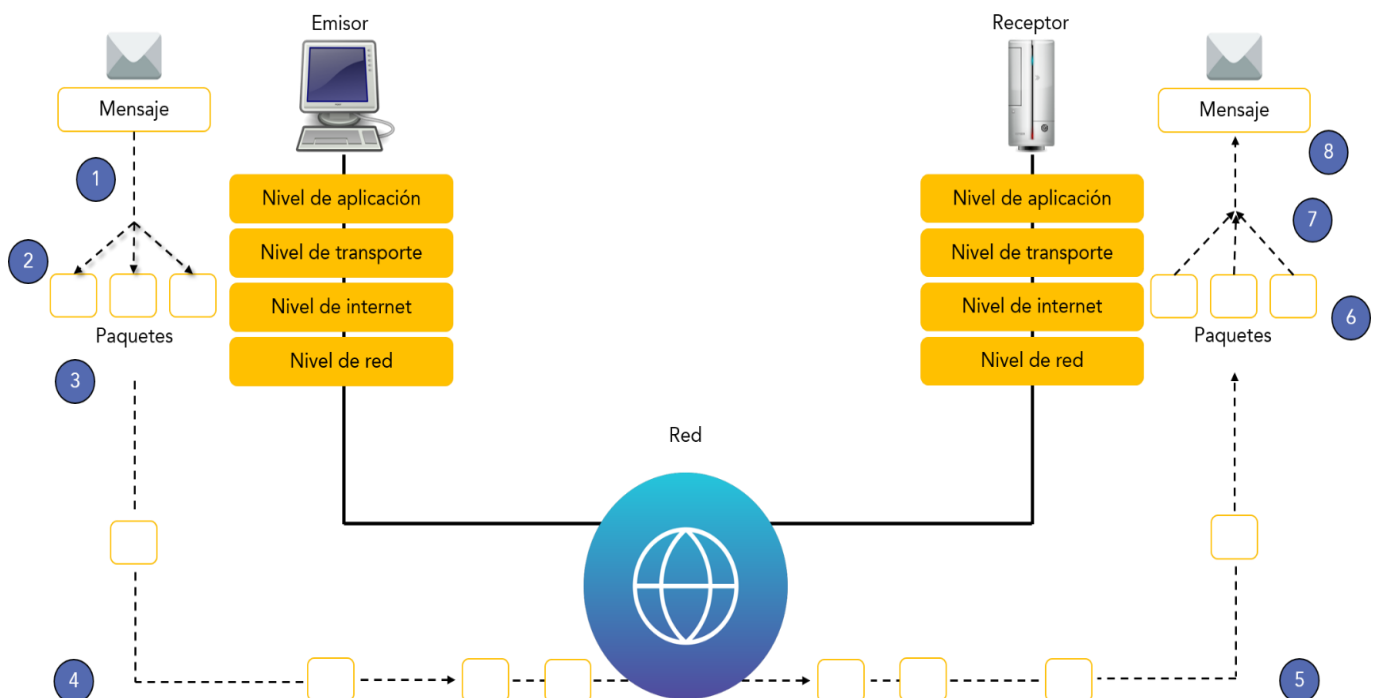
Para implementar estas redes, se utilizan unas tecnologías bastante complejas que dividen en capas para simplificar un poco las diferentes funciones que se deben llevar a cabo. Cada una de estas capas será la encargada de una tarea determinada, poniendo en práctica los diferentes recursos (software y hardware) disponibles.

A continuación, es posible ver que se presentan unas encima de otras, como si fueran una pila, de tal forma que, cada capa se puede comunicar con las que tiene encima y debajo. En el modelo TCP/IP, se divide la comunicación en cuatro capas.



Cuando una aplicación quiere enviar un mensaje se siguen los siguientes pasos:

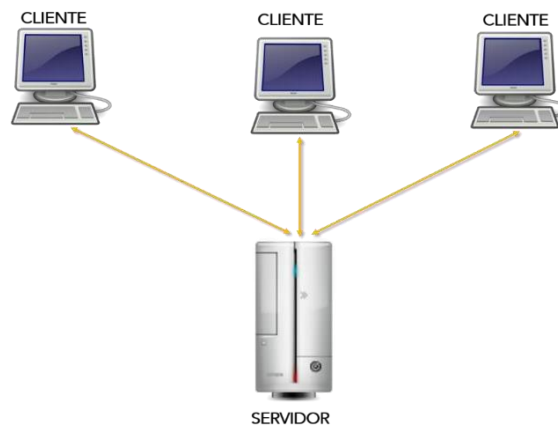
1. El emisor envía el mensaje, lo que significa que el mensaje pasa de la capa de aplicación a la capa de transporte.
2. En el nivel de transporte se divide el mensaje en paquetes para realizar el envío y lo pasa a la capa inferior, la capa de Internet.
3. En el nivel de Internet se comprueba el destino de los paquetes y se calcula la ruta que deben seguir; después se envían los paquetes al nivel de red.
4. El nivel de red se encarga de transmitir el paquete al receptor del mensaje.
5. El receptor recibe los paquetes en el nivel de red, en el más bajo, y los envía a la siguiente capa, que se corresponde con el nivel de Internet.
6. El nivel de Internet del receptor es el encargado de comprobar que son correctos. Si es así, los reenvía al nivel de transporte.
7. El nivel de transporte es el encargado de formar el mensaje con los paquetes recibidos y envía el mensaje a la última capa, el nivel de aplicación.
8. El nivel de aplicación recibe el mensaje correctamente.



1.2. Roles cliente y servidor

El modelo cliente-servidor se basa en una arquitectura en la que existen distintos recursos, a los que se denomina servidores, y un determinado número de clientes, es decir, de sistemas que requieren de esos recursos. Los servidores, además de proveer de recursos a los clientes, también ofrecen una serie de servicios que se estudiarán en el siguiente capítulo.

En este modelo, los clientes son los encargados de realizar peticiones a los servidores, y estos responden con la información necesaria.



En la actualidad existen distintas aplicaciones que utilizan este modelo, como pueden ser el correo electrónico, mensajería instantánea y el servicio de Internet, es decir, se utiliza siempre que se accede a una página web.



Se debe tener en cuenta que, en este modelo, no se intercambian los roles entre clientes y servidores.

Los sockets también son un ejemplo del modelo cliente-servidor.

1.3. Elementos de programación de aplicaciones en red. Librerías

- Clase InetAddress

Es la clase que representa las direcciones IP.

<code>byte[] getAddress ()</code>	Devuelve la dirección IP sin procesar como objeto
<code>static InetAddress getLocalHost ()</code>	Devuelve la dirección IP de la máquina
<code>static InetAddress getLocalHost (String host)</code>	Devuelve la dirección IP de la máquina especificada
<code>String toString ()</code>	Convierte la dirección IP en una cadena

Para más información:

<https://docs.oracle.com/javase/7/docs/api/java/net/InetAddress.html>

- Clase URL

Es la clase que representa un localizador de recursos uniforme (URL), es decir, representa la dirección del recurso en Internet.

Una URL es un conjunto de caracteres que permiten denominar de forma única los recursos en Internet, de esta forma facilita el acceso a ellos.

El formato de una URL es el siguiente:
`protocolo://máquina:puerto/ruta_fichero`

Si es necesario la identificación para acceder a ese recurso, el usuario y contraseña se deben indicar delante de la máquina, de forma que la URL quedaría:

`protocolo:/usuario:contraseña@máquina:puerto/ruta_fichero`

Existen distintas formas de crear un objeto de esta clase, dependiendo de los parámetros de los que se disponga. Por ello, existen diferentes constructores:

URL (String url)	Crea un objeto URL de la cadena recibida.
URL (String protocol, String host, int port, String file)	Crea un objeto URL con los datos recibidos.
URL (String protocol, String host, String file)	Crea un objeto URL con los datos recibidos.

Para más información:

<https://docs.oracle.com/javase/7/docs/api/java/net/URL.html>

Los métodos son:

String getFile ()	Devuelve el nombre del archivo
String getHost ()	Devuelve el nombre de la máquina
String getPath ()	Devuelve el nombre de la ruta de acceso
int getPort ()	Devuelve el puerto que ocupa
String getProtocol ()	Devuelve el protocolo
URI toURI ()	Devuelve el URI equivalente a esta URL
URLConnection.openConnection ()	Crea y devuelve una conexión al objeto remoto de esta URL

- Clase URLConnection

Esta clase permite trabajar con conexiones realizadas a la URL especificada. Para ello, es necesario crear un objeto de la clase URL e invocar al método `openConnection()`.

```
URL url = new URL ("https://https://www.colegiojosefinos.com/");  
URLConnection urlCon = url.openConnection();
```

Con esto obtenemos una conexión al objeto URL referenciado. Las instancias de esta clase se pueden utilizar tanto para leer como para escribir al recurso referenciado por la URL.

Algunos métodos de esta clase son:

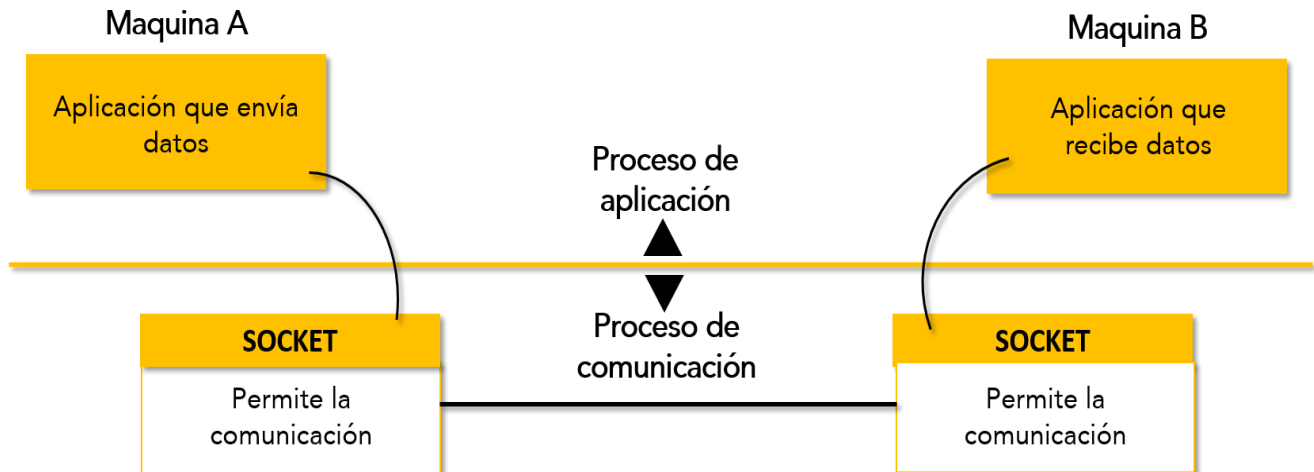
<code>abstract void connect ()</code>	Establece la conexión con el recurso de la URL. Una vez que se obtiene la conexión se pueden leer y escribir datos en dicha conexión
<code>InputStream getInputStream ()</code>	Devuelve un flujo de entrada para leer los datos del recurso
<code>OutputStream getOutourStream ()</code>	Devuelve un flujo de salida para escribir datos en el recurso
<code>URL getURL()</code>	Devuelve la URL de la conexión

Para más información:

<https://docs.oracle.com/javase/7/docs/api/java/net/URLConnection.html>

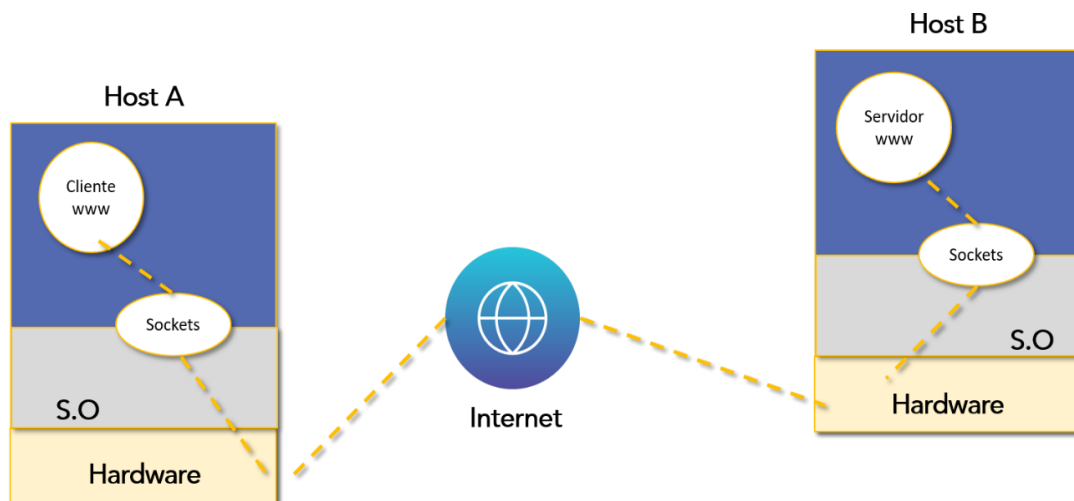
1.4. Sockets

Un socket es un mecanismo que permite la comunicación entre aplicaciones a través de la red, es decir, abstrae al usuario del paso de la información entre las distintas capas. Su función principal es crear un canal de comunicación entre las aplicaciones y simplificar el intercambio de mensajes.



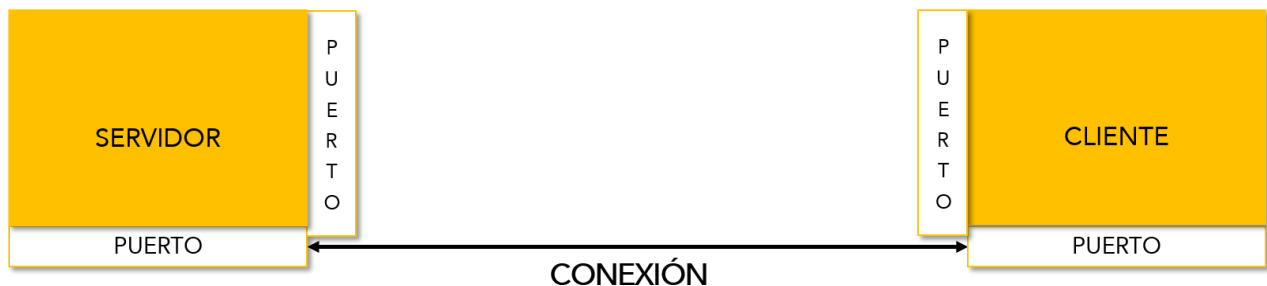
Un socket se define mediante la dirección IP de los dos dispositivos, el protocolo de transporte, y, por último, el puerto de cada uno de los dispositivos por el que se conectan. Un puerto es el identificador que permite la comunicación entre los dispositivos de una red. Una máquina solo puede tener un único puerto asignado.

La máquina denominada servidor tiene un puerto asignado, que es el encargado de quedarse a la espera de que algún cliente realice cualquier petición.



Para que un cliente pueda comunicarse con un servidor, debe conocer su dirección IP y el puerto asignado, y realizar la petición. Cuando el servidor la recibe, si decide aceptarla, asigna el puerto para la comunicación, de forma que el socket conocido por el cliente queda libre para recibir a nuevos clientes.

Como la petición la realiza desde el puerto del cliente, el servidor ya conoce su puerto, y es el cliente el que aceptará la conexión, y después, asignará el puerto para el envío de mensajes.



Existen dos tipos de sockets: los orientados a conexión y los no orientados a conexión.

- Sockets orientados a conexión

Este tipo de sockets utilizan el protocolo TCP. Se utilizan para aquellas aplicaciones que requieren una alta fiabilidad en el envío de mensajes, puesto que, mediante este protocolo, aseguran la entrega de todos y cada uno de los paquetes, en el mismo orden en el que fueron enviados. Para ello, se utiliza una verificación de los paquetes en el receptor, enviando un acuse de recibo. Si el emisor del mensaje no lo recibe, se procederá entonces al reenvío del paquete.

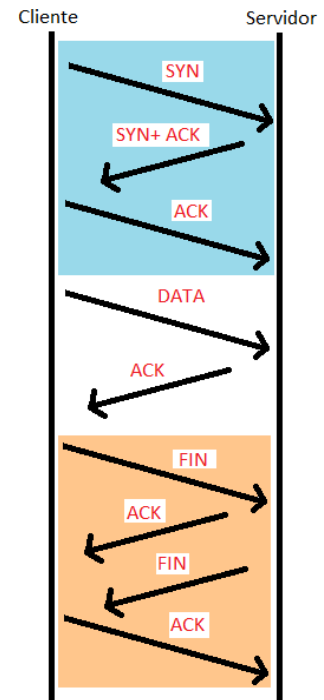
Otra de las principales características de este tipo de sockets es el establecimiento de la conexión. Para realizarla es necesario el envío de varios mensajes antes de comenzar el intercambio de mensajes:

- Petición del cliente de establecer conexión (SYN).
- Confirmación (ACK) por parte del servidor para establecer la conexión (SYN).
- Confirmación de cliente del mensaje anterior (ACK).

Una vez que estos tres mensajes han sido enviados, es posible empezar a enviar mensajes entre los dispositivos.

Cuando termina el envío de mensajes, el cliente es el que decide cerrar la conexión, para ello existen otro tipo de mensajes:

- El cliente envía el mensaje para cerrar la conexión (FIN).
- El servidor confirma el cierre de conexión (ACK).
- El servidor envía el cierre de la conexión (la otra vía) (FIN).
- El cliente confirma el cierre de conexión (ACK).



Los sockets orientados a conexión se utilizan en distintos servicios como, por ejemplo, FTP, Telnet, HTTP y SMTP que se estudiarán en el próximo capítulo.

- Sockets no orientados a conexión

Este tipo de sockets utilizan el protocolo UDP. Al contrario que los anteriores, este tipo de sockets no garantizan que los mensajes enviados lleguen a su destino, por lo que no es un protocolo fiable. Además, tampoco garantizan que los paquetes vayan a llegar en el orden enviado. A cambio de esto, ofrecen una mayor velocidad en el intercambio de los mensajes, puesto que no tienen que establecer conexión para ello, ni controlar los mensajes que han llegado.

Los sockets no orientados a conexión también son utilizados en los servicios. En este caso son utilizados por SNMP, DNS y NFS.

1.5. Utilización de sockets para la transmisión y recepción de información

En la capa de transporte hay que destacar dos protocolos, TCP y UDP, que son muy importantes en la comunicación en red.

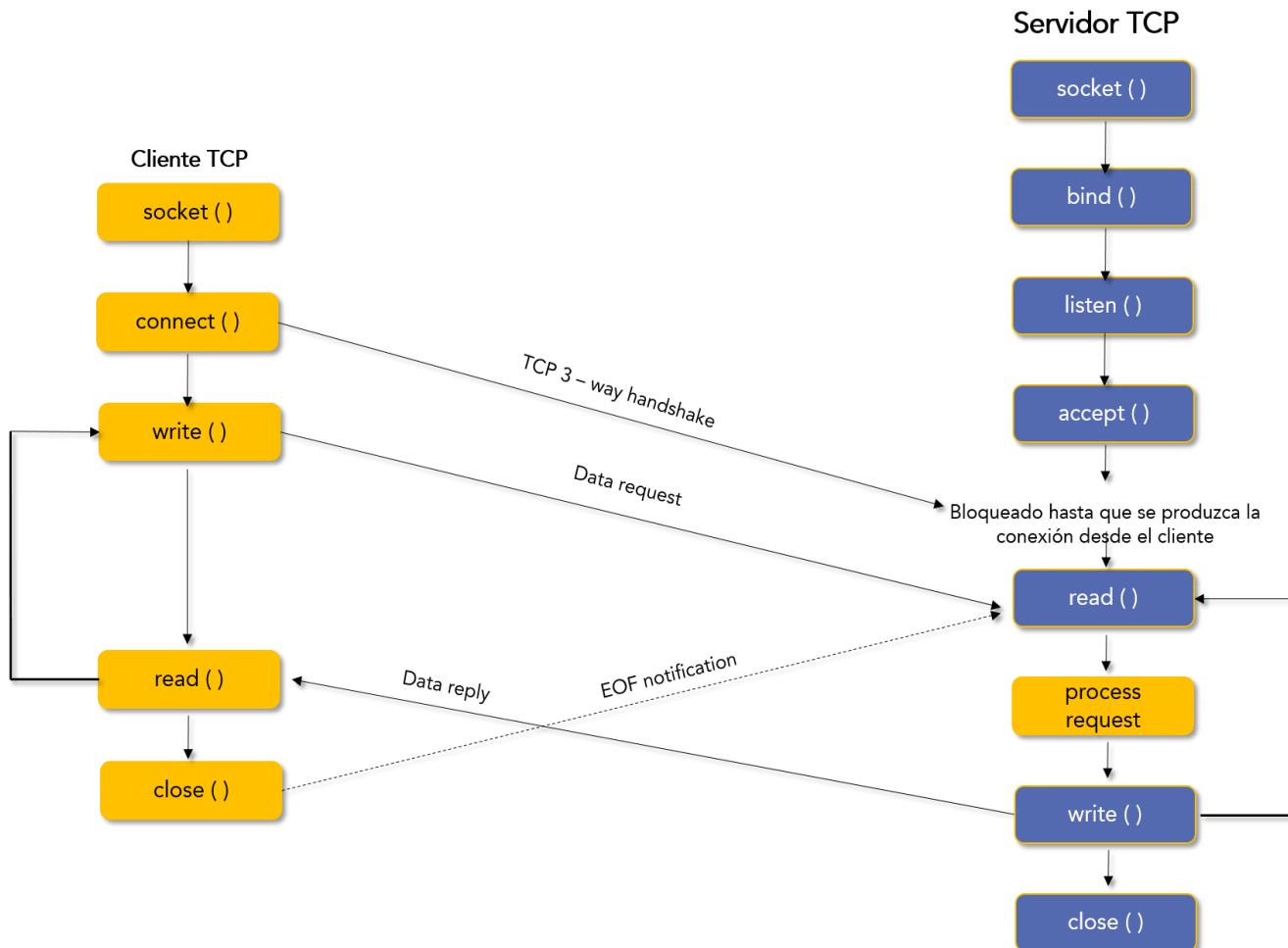
Aunque ambos sean protocolos de la capa de transporte, tienen una gran cantidad de diferencias:

	TCP	UDP
Conexión	Protocolo orientado a conexiones.	Protocolo sin conexiones.
Función	Se usa para enviar mensajes por Internet de una computadora a otra, mediante conexiones virtuales	Se usa para transporte de mensajes y/o transferencias. Al no estar basada en conexiones, un programa puede enviar una carga de paquetes y recibirse en el destino.
Uso	Aplicaciones que requieren confiabilidad alta y donde el tiempo de transmisión es menos crítico.	Aplicaciones que necesitan transmisión rápida y efectiva. Servidores que reciben una gran cantidad de peticiones pequeñas de un alto número de clientes.
Uso por otros protocolos	HTTP, HTTPS, SMTP, Telnet	DNS, DHCP, TFTP, SNMP, RIP, VoIP
Ordenar por paquetes de data	Orden especificado.	No tienen orden inherente. Los paquetes son independientes unos de los otros. Si requieren un orden, esto se maneja a nivel de aplicación.
Velocidad de transferencia	Más lento.	No hace falta verificación de errores por paquete, por lo que su velocidad es mayor.
Confiabilidad	Ofrece una garantía absoluta de que los datos llegarán en el mismo orden en que se enviaron.	No hay garantía de que los paquetes de datos lleguen.
Tamaño del Título	20 bits	8 bits
Campos comunes de títulos	Puerto de origen, puerto de destino, checksum.	

Fluidez	Los datos se leen como una secuencia de bits y no se transmiten indicadores para los límites de segmentos de los mensajes.	Los paquetes son enviados individualmente; se verifica su integridad solo si llegan. Los paquetes tienen límites definidos, que comprueban que el mensaje está completo.
Peso	TCP es pesado. Requiere tres paquetes para establecer una conexión antes de transmitir. TCP maneja confiabilidad y control de congestión.	UDP es liviano. No hay ordenamiento de mensajes ni conexiones de verificación.
Control de flujo de data	Sí realiza control de flujo. Requiere tres paquetes para establecer una conexión antes de transmitir. Maneja confiabilidad y control de congestión.	No se realiza control de flujo.
Verificación de errores	Sí hay verificación de errores.	Sí hay verificación de errores, pero no tiene opciones para recuperar los datos perdidos.
Campos	1. Número de secuencia 2. Número de ACK 3. Índice data 4. Reservado 5. Bit de control 6. Ventana 7. Indicador de urgencia 8. Opciones 9. Relleno 10. CheckSum 11. Puerto de origen 12. Puerto de destino	1. Largo 2. Puerto de origen 3. Puerto de destino 4. CheckSum
Reconocimiento	Hay segmentos de reconocimiento.	No hace reconocimiento.
"Handshake" (verifica conexiones en tres tiempos)	SYN SYN-ACK ACK	No hace esta verificación.
Checksum	Completo	Solo para detectar errores.

1.6. Creación de sockets

- Sockets orientados a conexión



- Clase ServerSocket

Es la clase que se debe instanciar en la parte del servidor para crear el puerto que se queda esperando a la conexión por parte de los clientes.

Existen distintos constructores:

<code>ServerSocket ()</code>	Crea un socket no enlazado.
<code>ServerSocket (int port)</code>	Crea un socket enlazado al puerto especificado.
<code>ServerSocket (int port, int backlog)</code>	Crea un socket enlazado al puerto especificado, indicando el número máximo de peticiones que pueden estar en cola.

ServerSocket (int port, int backlog, InetAddress dirección)

Crea un socket enlazado al puerto especificado y a una dirección IP, indicando el número máximo de peticiones que pueden estar en cola.

Para más información:

<https://docs.oracle.com/javase/7/docs/api/java/net/ServerSocket.html>

- Clase Socket

Es la clase que se debe instanciar en la parte del cliente.

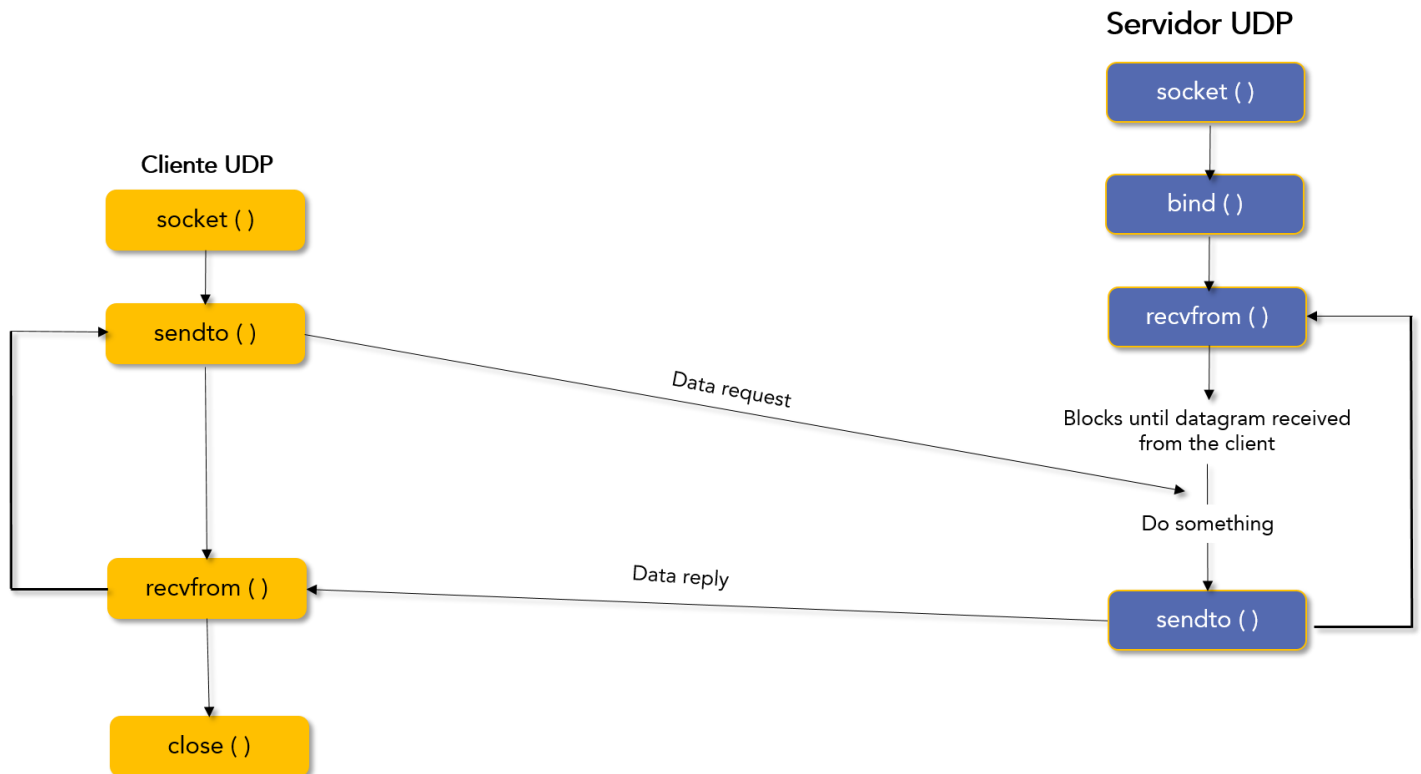
Existen distintos constructores:

Socket ()	Crea un socket no conectado.
Socket (InetAddress address, int port)	Crea un socket conectado al puerto indicado en la dirección IP especificada.
Socket (InetAddress address, int port, InetAddress localAddr, int localPort)	Crea un socket y lo conecta a la dirección remota especificada en el puerto remoto especificado.
Socket (String host, int port)	Crea un socket y lo conecta al puerto especificado en el host.
Socket (String host, int port, InetAddress localAddr, int localPort)	Crea un socket y lo conecta al host remoto especificado en el puerto especificado.

Para más información:

<https://docs.oracle.com/javase/7/docs/api/java/net/Socket.html>

- Sockets no orientados a conexión



- Clase DatagramSocket

Es la clase que se debe instanciar tanto en la parte del cliente como en el servidor.

Existen distintos constructores:

<code>DatagramSocket ()</code>	Crea un socket UDP y lo conecta a cualquier puerto disponible de la máquina host remota.
<code>DatagramSocket (int port)</code>	Crea un socket UDP y lo conecta con el puerto especificado.
<code>DatagramSocket (int port, InetAddress laddr)</code>	Crea un socket UDP y lo conecta a la dirección local especificada.
<code>DatagramSocket (SocketAddress bindaddr)</code>	Crea un socket UDP y lo conecta a la dirección de socket local especificada.

Para más información:

<https://docs.oracle.com/javase/7/docs/api/java/net/DatagramSocket.html>

- Clase DatagramPacket

Es la clase que permite crear paquetes para enviarlos a través del socket UDP.

Existen distintos constructores:

DatagramPacket (byte [] buf, int longitud)	Crea un datagrama que recibe paquetes de la longitud especificada.
DatagramPacket (byte [] buf, int longitud, InetAddress address, int port)	Crea un datagrama que envía paquetes de la longitud especificada al puerto especificado.
DatagramPacket (byte [] buf, int offset, int longitud)	Crea un datagrama que recibe paquetes de la longitud especificada del puerto especificado.
DatagramPacket (byte [] buf, int longitud, SocketAddress address)	Crea un datagrama que envía paquetes de la longitud especificada al host especificado.

Para más información:

<https://docs.oracle.com/javase/7/docs/api/java/net/DatagramPacket.html>

1.7. Enlazamiento y establecimiento de conexiones

- Sockets orientados a conexión
- Clase ServerSocket

Algunos de sus métodos más importantes:

Socket accept ()	Acepta una petición de conexión por parte del cliente.
void bind (SocketAddress endpoint)	Vincula el socket a una dirección IP y a un puerto determinado.
void close ()	Cierra el socket.
InetAddress getInetAddress ()	Devuelve la dirección local del socket.
int getPort ()	Devuelve el puerto del socket.

Para más información:

<https://docs.oracle.com/javase/7/docs/api/java/net/ServerSocket.html>

- Clase Socket

Algunos de sus métodos más importantes:

<code>void bind (SocketAddress endpoint)</code>	Vincula el socket a una dirección local.
<code>void close ()</code>	Cierra el socket.
<code>void connect (SocketAddress endpoint)</code>	Conecta el socket con el servidor.
<code>InputStream getInputStream ()</code>	Devuelve el flujo de entrada para el socket.
<code>InetAddress getLocalAddress ()</code>	Devuelve la dirección a la que está conectado el socket.
<code>int getLocalPort ()</code>	Devuelve el número de puerto local al que está vinculado el socket.
<code>OutputStream getOutputStream ()</code>	Devuelve el flujo de salida para el socket.
<code>int getPort ()</code>	Devuelve el número de puerto remoto al que está conectado el socket.
<code>boolean isClosed ()</code>	Comprueba si está cerrado el socket.
<code>boolean isConnected ()</code>	Comprueba si está conectado el socket.

Para más información:

<https://docs.oracle.com/javase/7/docs/api/java/net/Socket.html>

- Servicios no orientados a conexión
- Clase DatagramSocket

Algunos de sus métodos más importantes:

<code>void bind (SocketAddress addr)</code>	Vincula el socket a una dirección y puerto especificado.
<code>void close ()</code>	Cierra el socket.
<code>void connect (InetAddress address, int port)</code>	Conecta el socket a una dirección remota.
<code>void disconnect ()</code>	Desconecta el socket.
<code>InetAddress getInetAddress ()</code>	Devuelve la dirección local a la que está vinculado el socket.

<code>int getPort ()</code>	Devuelve el número de puerto al que está conectado este socket.
<code>void receive (DatagramPacket p)</code>	Recibe un paquete de datagramas del socket.
<code>void send (DatagramPacket p)</code>	Envía un paquete de datagramas al socket.
<code>setSoTimeout (int time)</code>	Habilita o deshabilita un tiempo de espera especificado, en milisegundos.

Para más información:

<https://docs.oracle.com/javase/7/docs/api/java/net/DatagramSocket.html>

- Clase DatagramPacket

Algunos de sus métodos más importantes:

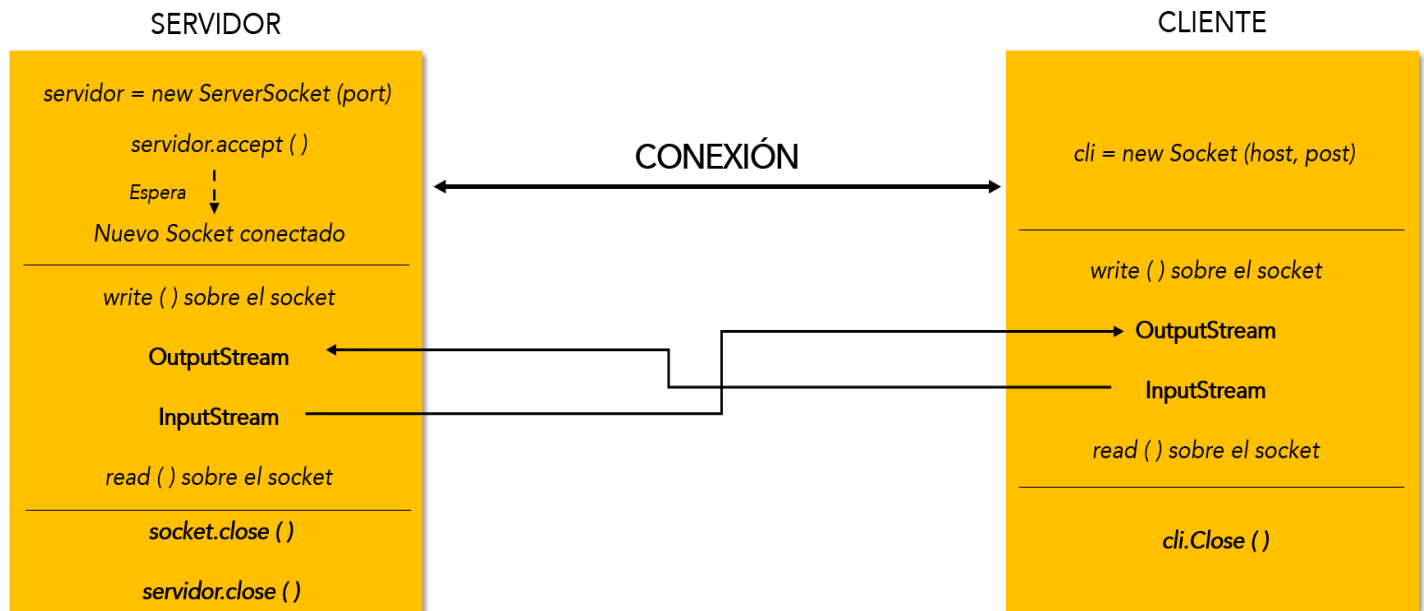
<code>InetAddress getAddress ()</code>	Devuelve la dirección IP de la máquina con la que se ha realizado una transmisión de paquetes.
<code>byte [] getData ()</code>	Devuelve el búfer de datos.
<code>int getLength ()</code>	Devuelve la longitud de los datos.
<code>int getPort ()</code>	Devuelve el puerto del host remoto con el que se ha realizado una transmisión de paquetes.
<code>void setAddress (InetAddress address)</code>	Establece la dirección IP de la máquina a la que se envía este datagrama.
<code>void setData (byte [] data)</code>	Establece el búfer de datos para un paquete.
<code>void setLength (int longitud)</code>	Establece la longitud del paquete.
<code>void setPort (int port)</code>	Establece el puerto del host remoto al que se envía el datagrama.

Para más información:

<https://docs.oracle.com/javase/7/docs/api/java/net/DatagramPacket.html>

1.8. Programación de aplicaciones cliente y servidor

- Socket TCP

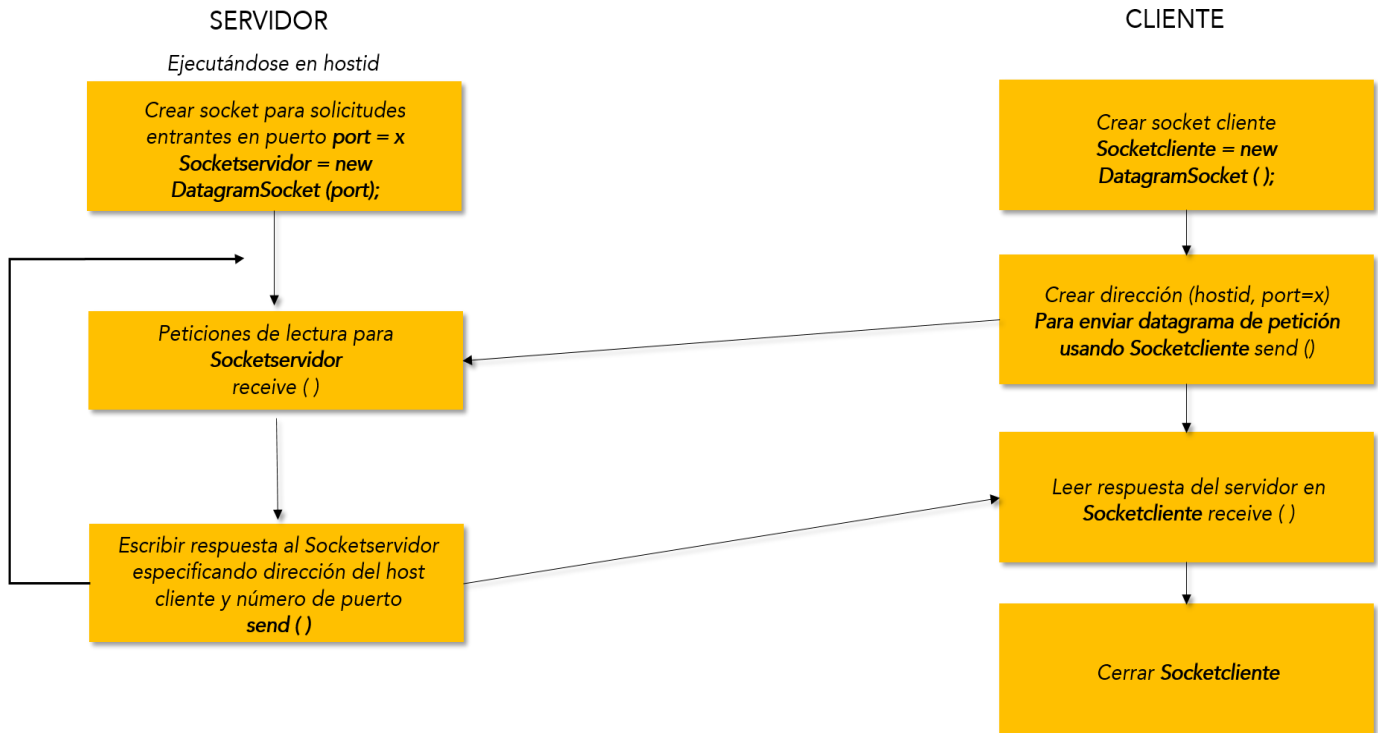


Cuando se crea un socket en el servidor, se queda esperando a que los clientes se conecten. Esto se realiza mediante la función `accept()`. Cuando se crea un socket en el cliente, indicando el puerto y la dirección del socket servidor, se realiza la conexión entre el cliente y el servidor.

Una vez que existe conexión, comienza la transmisión de los datos, mediante funciones de `write` y `read`. Estas operaciones se realizan mediante las clases `DataInputStream` y `DataOutputStream`, que permiten utilizar diversos métodos de lectura y de escritura.

Cuando se termina la transmisión de los datos, se cierra la conexión, y también el socket del cliente. Después, cuando el servidor acaba su función, también se cierra.

- Socket UDP



Se crea un socket en el servidor que se queda a la espera de peticiones de clientes. Cuando se crea un socket en el cliente, se conecta al socket del servidor. Una vez realizada la conexión se envían los distintos datagramas mediante los métodos send y receive.

Cuando se termina la transmisión de los datos, se cierra el socket del cliente. El servidor se puede quedar a la espera de otros clientes, pero cuando el servidor termina su función, también hay que cerrar su socket.

1.9. Utilización de hilos en la programación de aplicaciones en red

Mediante los sockets creados hasta ahora, el servidor únicamente es capaz de trabajar con un cliente simultáneamente. La solución para esto pasa por la creación de hilos para contestar a cada cliente. Es decir, cuando se crea el socket del servidor y se queda esperando clientes y recibe la petición de un cliente, debe crear un Hilo con el resto del funcionamiento del servidor.

De esta forma, el servidor podrá atender a todos los clientes que realicen la petición.

